Next-Phase AI Roadmap

Consolidated AI Development Documentation — Day 15–20 Review and Day 21+ Plan

Purpose

This document consolidates the AI work discussed previously and turns it into a practical next-phase development plan. It covers the earlier AI feasibility research, RAG planning, AI Content Recommendations, FAQ Assistant, chatbot, multilingual support, Financial AI, Smart Insights, User Engagement, milestones, ownership, dependencies, testing, GitHub workflow and long-term scalability.

Status rule: A feature is marked Planned unless previous work clearly confirmed its implementation. This avoids claiming that an unverified module is already complete.

1. Review of Previous AI Work

- **AI Feasibility Research:** technical feasibility, AI model research, Ollama feasibility, RAG feasibility, FastAPI integration, offline-first possibilities and cost considerations.
- **AI Content Recommendations:** personalized content suggestions, related-content recommendations and user-behavior-based recommendations.
- **Next-Phase AI Roadmap:** RAG, FAQ Assistant, chatbot, multilingual support, Financial AI, Smart Insights, recommendations, engagement, milestones, ownership, dependencies, testing and GitHub workflow.
- **Application context:** earlier AI planning was discussed for the Hisabdo application, including financial-data-driven AI capabilities.

2. Current AI Module Status

Module / Area	Status	Notes
AI feasibility research	Reviewed / Completed	Previous research covered technical feasibility and implementation options.
AI model research	Reviewed / Completed	Cloud and local model approaches were considered.
Ollama feasibility	Reviewed	Potential local/offline model option for future use.
RAG knowledge base	Planned	Core next-phase knowledge layer.
AI Content Recommendations	Designed / Planned	Personalized, related and behavior-based recommendations.
FAQ Assistant	Planned	AI answers for common application questions.
AI Chatbot	Planned	Conversational interface over the AI services.
Multilingual support	Planned	Multiple-language input and output.
Financial AI	Planned	Financial analysis and explanations.
Smart Insights & Recommendations	Planned	Convert data into useful personalized insights.
User Engagement	Planned	Useful AI-driven prompts and recommendations.

3. Priorities for Day 21 Onward

- **Priority 1:** RAG Knowledge Base — create the reliable knowledge foundation.
- **Priority 2:** FAQ Assistant — answer common questions from approved information.
- **Priority 3:** AI Chatbot — provide a central conversational interface.
- **Priority 4:** Multilingual Support — return answers in the user's selected language.
- **Priority 5:** Financial AI — analyze financial information safely.
- **Priority 6:** Smart Insights & Recommendations — generate personalized value from data.

- **Priority 7:** User Engagement — use AI to improve useful product interaction.
- **Priority 8:** Optimization — improve accuracy, speed, security, cost and scalability.

4. RAG Knowledge Base Development

RAG (Retrieval-Augmented Generation) lets the system retrieve relevant information first and then give the AI that information as context. This reduces unsupported answers and makes the assistant more grounded in approved application/business data.

```
Flow: User Question → Backend → Query/Embedding → Vector Store → Relevant Documents → LLM → Grounded Answer → User
```

- Collect approved documentation, FAQs and business rules.
- Clean and split documents into useful chunks.
- Generate embeddings and store them with metadata.
- Retrieve the most relevant chunks for each question.
- Send retrieved context to the model.
- Add source/reference information where appropriate.
- Evaluate retrieval accuracy and hallucinations.

5. FAQ Assistant Implementation

The FAQ Assistant will answer common questions using the FAQ/RAG knowledge base.

- Maintain an approved FAQ dataset.
- Search/retrieve the relevant answer context.
- Return short and understandable answers.
- Use a safe fallback when information is unavailable.
- Log questions and answers for quality evaluation.

6. AI Chatbot Integration

The chatbot should be a frontend interface connected to a backend AI service. Sensitive model logic, authentication and data access should remain on the backend.

```
Architecture: Chat UI → Backend AI API → Authentication/Validation → RAG / Financial AI / Recommendation Services → LLM → Response
```

- Conversation history where needed.
- User authentication and authorization.
- RAG integration.
- Financial-data access only for authorized users.
- Timeout and error handling.
- Usage and quality monitoring.

7. Multilingual Support

- Allow language selection or detect the user's language.
- Keep business logic and data processing language-independent.
- Generate the final answer in the selected language.
- Keep financial numbers, dates and currency values accurate.
- Test the same questions across all supported languages.

8. Financial AI Integration

Financial AI should analyze and explain application data. The application/backend should remain the source of truth for calculations; the AI should explain results and produce insights.

- Income and expense summaries.
- Category-wise spending analysis.
- Budget comparison.
- Unusual spending detection.
- Period-over-period comparisons.
- Cash-flow summaries.
- Plain-language financial explanations.
- Actionable recommendations.

9. Smart Insights & Recommendations

This extends the previously discussed AI Content Recommendations work.

- **Personalized recommendations:** based on the user's own activity.
- **Related content:** show information related to the current content/question.
- **Behavior-based recommendations:** use meaningful usage patterns.
- **Smart financial insights:** identify trends and areas needing attention.
- **Explainability:** where possible, tell the user why an insight/recommendation was generated.

10. User Engagement Implementation

- Personalized reminders or prompts.
- Relevant recommendations.
- Progress/activity summaries.
- Helpful chatbot follow-ups.
- Optional notifications based on user preferences.
- Measure whether engagement features provide real user value.

11. Development Milestones

Milestone	Area	Main Deliverable
1	Foundation	Architecture, model strategy, security and data requirements.
2	RAG	Ingestion, embeddings, retrieval and evaluation.
3	FAQ	Reliable FAQ/RAG answer workflow.
4	Chatbot	Conversational UI and backend AI integration.
5	Multilingual	Language handling and evaluation.
6	Financial AI	Deterministic analytics plus AI explanations.
7	Insights	Smart insights and recommendation logic.
8	Engagement	Personalized AI engagement features.
9	Quality	Security, performance, accuracy and regression testing.
10	Scale	Monitoring, caching, cost control and production readiness.

12. Ownership for Next-Phase Modules

Assign one clear owner per module to prevent duplicate work. Actual names can be filled in by the team lead.

Module	Owner	Responsibility
RAG Knowledge Base	Assign	Ingestion, retrieval and evaluation.
FAQ Assistant	Assign	FAQ data and answer/fallback flow.
AI Chatbot	Assign	Chat UI and AI backend integration.
Multilingual	Assign	Language handling and testing.
Financial AI	Assign	Financial analytics integration and explanations.
Recommendations	Assign	Personalization and recommendation logic.
Testing/Evaluation	Assign	AI test cases, datasets and regression.
GitHub/DevOps	Assign	Branches, CI/CD, environments and deployment.

13. Technical Dependencies & Blockers

- **Model:** hosted LLM or local model such as an Ollama-supported model.
- **RAG storage:** vector database/vector-capable storage and embeddings.
- **Backend:** FastAPI or the project's existing backend API layer.
- **Data:** clean documentation, FAQs and authorized application data.
- **Authentication:** user identity and permissions.
- **Infrastructure:** development, staging and production environments.
- **Cost/limits:** token usage, latency, quotas and hosting cost.
- **Security:** protect sensitive financial information and secrets.
- **Evaluation data:** representative questions with expected answers.

14. Testing & Evaluation Strategy

Test Area	Question	Approach
Functional	Does the feature work?	UI, API, auth, permissions and error testing.
RAG retrieval	Did it find the right context?	Known questions with expected relevant documents.
Answer accuracy	Is the answer correct?	Compare against approved expected answers.
Hallucination	Does it invent information?	Unsupported questions and safe-fallback tests.
Financial accuracy	Are numbers correct?	Compare with deterministic backend calculations.
Multilingual	Is meaning preserved?	Test terminology, meaning and financial values.
Performance	Is it fast enough?	Measure response time and concurrent requests.
Security	Is data protected?	Authorization, prompt-injection and sensitive-data tests.
Regression	Did changes break old behavior?	Run fixed AI evaluation suite after changes.

15. GitHub Development Workflow

- Create one feature branch per AI module, for example feature/rag or feature/financial-ai.
- Use small and meaningful commits.
- Open a Pull Request for review.
- Run automated tests before merging.
- Require code review for production branches.
- Merge only after tests and review pass.

- Tag stable milestones/releases.
- Never commit API keys or environment secrets.

16. Scalable Long-Term AI Vision

The long-term goal is a modular AI platform where new AI capabilities can be added without rebuilding the whole application.

Target architecture: Web/Mobile App → API Gateway → AI Orchestration Layer → {RAG, FAQ, Chatbot, Financial AI, Recommendations} → Model Layer → Data/Vector Stores → Monitoring & Evaluation

- Keep AI services modular.
- Separate business logic from prompts.
- Use RAG for controlled knowledge.
- Keep deterministic calculations outside the LLM.
- Support cloud and local model options where practical.
- Use caching/asynchronous processing when useful.
- Monitor quality, latency, errors and cost.
- Version prompts, evaluation datasets and model configurations.
- Protect user data through authentication and secure logging.
- Design APIs so additional models can be introduced later.

17. Overall Implementation Flow

Existing Application Data → Data Cleaning & Knowledge Preparation → RAG Knowledge Base + Financial Data Services → AI Orchestration/Backend → LLM → FAQ / Chatbot / Financial AI / Recommendations → Multilingual Response + Smart Insights → User Engagement → Monitoring + Testing + Continuous Improvement

18. Final Deliverables

- Current AI status matrix.
- Prioritized Day 21+ plan.
- RAG architecture and implementation plan.
- FAQ Assistant plan.
- AI Chatbot integration plan.
- Multilingual support plan.
- Financial AI plan.
- Smart Insights and Recommendations plan.
- User Engagement plan.
- Development milestones and ownership matrix.
- Dependencies and blockers.
- Testing and evaluation strategy.
- GitHub development workflow.
- Scalable long-term AI vision.

19. Conclusion

The next phase should move the project from AI research and planning toward a structured, testable AI platform. RAG should be the core knowledge foundation, followed by the FAQ Assistant and chatbot. Multilingual support, Financial AI, Smart Insights, Recommendations and User Engagement can then build on the same AI service layer. This reduces

duplication, improves maintainability and makes future AI features easier to add.